CSC111 Project 2 Phase 2: Final Submission

Note: Any FAQs or clarifications relevant to the project will be posted here (https://edstem.org/us/courses/89026/discussion/7774266). This post will be continually updated (with newer updates at the bottom of the page), so make sure to check on it regularly—click the "Watch" button in the top-right corner to receive notifications about this thread. If you have a question which is not already addressed on this page, create a new thread to post your question on Ed.

This course project is an opportunity to use what you have learned in this course and apply it in a creative, open-ended project. In Phase 1 of the project (../phase1/), you submitted a proposal describing your plan for a project that applies what you've learned about trees and/or graphs to a new problem domain of your choice. Now, in the second and final phase of the project, you will submit a project report and the program that you created. While this is the "final" submission of your project for CSC111, we encourage you to continue with the project even after the course is done and you learn new things!

Logistics

- Due date: Monday March 30, 1pm Toronto Time.

  Warning: the maximum extension you can get on this phase of the project is 3 days, not 7.

- For phase 2 you will be in the same groups as the proposal, and this will already be set up on MarkUs for the phase 2 submission. Please contact us if you would like to change groups.

- You will submit your project on MarkUs (see submission instructions at the end of this handout).

- General expectations, including late policy and extensions: see the Assignment Expectations page (https://q.utoronto.ca/courses/418277/pages/assignment-expectations) and the syllabus.
- This project must be done within your group, with no collaboration outside of it. Please review the Academic Integrity policies in the syllabus.

Publishing your project

The syllabus states that course work, like assignments and projects, cannot be published publicly. Since this project is open-ended, there is an exception for this project. However, **project 2 should not be published publicly online until after the course ends**, after the end of the exam period.

Your other assignments and project 1 should not be posted publicly, as stated in the syllabus.

Project overview

Before continuing to the rest of this handout, we recommend reviewing the Project theme and other instructions from Phase 1 (Proposal) (../phase1/#project-theme-applications-of-trees-and-graphs). We have not repeated them here.

Your project submission is divided into two components, a **written report** and the **Python program** you created for this project. The next two sections of this handout describes the required elements for both of these components.

1. Your written report

Your project written report builds on your project proposal from Phase 1, incorporating any changes to your project direction, a more detailed computational plan, and a discussion of the results of your exploration.

You should submit two files for this part: `project_report.pdf` and `project_report.tex`. We haven't provided a LaTeX template for this part, and instead encourage you to start with your `project_proposal.tex` file from Phase 1 and adapt it

for this phase.

Unlike the project proposal, you can and should also submit any images you use to create your report (e.g., using `\includegraphics`*).*

Your report must include the following sections:

1. **Project title** (pick something informative and professional, but you can be creative too) and **names** of all group members.

2. An **introduction** containing your problem description and project question/goal. You can start with the text from your project proposal, but then incorporate TA feedback or changes that your group has decided on. *It's okay if your question/goal (or entire project direction) has changed from what you originally proposed.* Review the original proposal instructions for this section to include necessary background information and motivation.

   As in the project proposal, your project question/goal should be in **bold**.

   Note: your report, and in particular this section, should also use [inline references (https://advice.writing.utoronto.ca/using-sources/documentation/)](https://advice.writing.utoronto.ca/using-sources/documentation/) when citing facts, data, or work that is not your own. You may use any academic reference style you wish for this project.

3. (*if applicable*) The name and description of **all datasets** you used for your project. If you generated your own datasets for your project, you should describe those in this section. Include the format and source of each dataset.

   When using large datasets, it is common to use only a subset of the data found within them. So this section should also state which parts of each dataset (e.g., which columns in a csv file) are actually used by your program.

   See #5 below about requirements for including datasets in your submission.

4. A **computational overview** for your project.

   This is similar to the computational plan you submitted in the proposal, except now it's not a plan, but a description of the program you're actually submitting. This section should:

   - Describe the kinds of data your project uses to represent your chosen domain, and how *trees and/or graphs play a* ***central*** *role* in this data representation.

- Describe the major the computations your program performs, such as: building trees/graphs from a dataset or computation, data transformation/filtering/aggregation, computational models, and/or algorithms.
- Explain how your program reports the results of your computation in a visual and/or interactive way.
- Explain how your program uses *new Python libraries* to accomplish its tasks. Refer to specific functions, data types, and/or capabilities of the library that make it relevant for accomplishing these tasks.

This overview should be more detailed and concrete than the proposal you submitted, since you are submitting your actual code as well. Refer to concrete files, data types, and functions that you created in your descriptions of this part.

However, you do not need to mention every single data type/function you wrote; use your judgment to only refer to the most important data types and functions that you created for each "computational phase" of your program. Your helper functions and data types should be documented in the code itself, but don't need to be discussed in your report.

5. Instructions for **obtaining datasets and running your program**. Your TA will expect to be able to run your program starting from just their computer with only Python (version 3.13.5) installed. You can expect that the TA will do the following:

   a. Install all Python libraries listed under a `requirements.txt` file that you submit as part of your project.

      - Your project must be "pure Python"—you may **not** use other programming languages or non-Python software, and your TA will not install these.
        - *Exception*: if you are making a simple interactive webpage to display your results, you may use **very small amounts** of Javascript code to make the webpage interactive. However, your TA won't be reviewing this code, nor take it into account when assessing your project's overall computational complexity.
      - If any Python libraries requires special installation instructions, include those instructions in your final report. If the installation procedure is especially complex (e.g. more than just installing through PyCharm), ask your instructor about this on Ed or in person.

b. Download datasets **from a zip file you provide**. Your TAs will *not* be downloading dataset files from external URLs, although you should still include the source URLs in your dataset description section earlier in your project report.

   Instead, for this project you must prepare a single zip file that contains all datasets used by your program, including ones that your program has pre-processed and modified, or has generated from scratch (if applicable).

   In this part of your project report, include instructions on how to download this zip file for your TA. Here are the three options for doing so:

   1. You've uploaded the zip file to MarkUs (**file size under 10 MB only**) as part of your final project submission.
   2. You've shared the zip file using https://send.utoronto.ca (https://send.utoronto.ca) (share it with the course email address, csc111-2026-01@cs.toronto.edu).
   3. You've uploaded the zip file to the U of T OneDrive (https://easi.its.utoronto.ca/initiatives/digital-workplace-intune-migration/onedrive/), and have an access link that your TA can use to download the file. *Your TA must be able to access this link without being logged into an external service (e.g., Google).* Requiring UTORid login is okay.

c. Run a file that you submit called `main.py`. (See next section for expectations for structuring your code.)

   - Describe what your TA should expect to see as a result of running your program (i.e., what the output looks like, what data is being shown, etc.) Screenshots are helpful here!
   - If your program produces an interactive visualization or display, describe the different interactive features and how your TA can use them.

6. A section describing **changes to your project plan** between your proposal and final submission. This can be based on TA feedback you received on your proposal, discussions with your instructor during office hours, or discussions with your group members or other ideas you wanted to explore.

7. A **discussion section** where you analyze and interpret the results of your program. Remember that the purpose of your project was to answer the research question or achieve a project goal you posed at the start of this report! Consider questions such as:

- Do the results of your computational exploration help answer this question/achieve your goal?
- What limitations did you encounter, with the datasets you found, the algorithms/libraries you used, or other obstacles?
  - *It's okay if your program's results do not seem very impressive*. What we care about is the computational strategies/algorithms you tried, how your program's results are displayed to the user, and how you analyze those results in this section.
- What are some next steps for further exploration?

You do *not* need a separate "Conclusion" section, so you may also choose to summarize and write a brief concluding paragraph at the end of this section.

8. A **references section** that lists the references you used for your project. This should include references from your topic research, references for every dataset you used, and any online documentation or tutorials you consulted for the Python libraries and algorithms you used for this project.
   - You may use any academic reference style you wish, e.g. MLA or IEEE.
   - See also this Overleaf tutorial (https://www.overleaf.com/learn/latex/Bibliography_management_in_LaTeX) on including references in LaTeX. This is optional, but you may find it interesting to dive deeper into LaTeX.

Note about word limits

There are no formal word limits or requirements for this report. The expected length for each section is comparable to but longer than the corresponding sections on your proposal, including more details (e.g., more datasets to describe, and a more concrete computational overview).

Your instructions to your TA for running your program should be clear and concise.

Your discussion section should be around 500-800 words.

2. Your complete program

For this project, you have a large amount of freedom in how you organize, design, and implement your code. Because of the open-ended nature of this project, we aren't running any automated testing on your submissions! Instead, you are submitting your code so that your TAs can assess the overall quality of your code, and their ability to actually run your program and reproduce the results that you report.

Code quality: organization, design, documentation, and style

Your program is the largest and most complex body of Python code you've written on your own (without any provided starter code) in this course. You should split up your program into multiple files organized into logical groups (e.g., the code responsible for reading data from data sets; the code responsible for computing on the data; the code responsible for displaying the results of the computations). You should also have one separate file called `main.py`, whose *main block* consists of the code necessary to run your entire program from start to finish.

Within each file, you are responsible for designing and implementing Python classes and functions to organize your data and operations on that data.

All Python files and code you submit must follow good design, documentation, and style principles that we've learned throughout this course.

1. *Modules*. Each module should have a title and description, following a similar format to the ones we've provided for you in starter files for preps and assignments. You and your group members should claim copyright on your own work!

   Modules should consist only of import statements, function and data type definitions, and a main block at the bottom of the file. There should not be any top-level statements/function calls outside of the main block.

2. *Functions and classes*. Every function and class that you define should follow the course standards for naming, headers, docstrings (including preconditions/representation invariants). This includes helper functions.

Doctest examples are not required, but strongly encouraged where possible to help your TA understand your code. You may, but are not required, to submit other tests you write for your project.

A convention we encourage, but do not require, you to follow is to list your functions in top-down order of importance: the "main" function(s) at the top of the file, and helper functions beneath them.

3. *Code-checking tools*. In each file except `main.py`, your main block should run `doctest.testmod()` to check any doctests, and use `python_ta.check_all` to check your work. You may include the `@check_contracts` when developing your code, but you should comment it out for your final submission to improve the running time of your code.

   Unlike past assessments, you may modify the `config` argument for `python_ta.check_all` to suit the needs of your module, e.g., by adding additional imports or listing function names that call `open` or `print` under the `'allowed-io'` option. We aren't automatically giving a grade based on passing all doctests or counting the number of PythonTA errors. However, your TAs will be reviewing both doctests and your overall code quality, and these tools are designed to help you check both of these—use them!

   Here is a sample call to `python_ta.check_all` that you can base your own configuration on:

```
python_ta.check_all(config={
    'extra-imports': [],  # the names (strs) of imported
        modules
    'allowed-io': [],     # the names (strs) of functions that
        call print/open/input
    'max-line-length': 120
})
```

Program requirements and reproducibility

As we discussed in the "written report" section above, your TA will expect to be able to run your program starting from just their computer starting with only Python (version 3.13.5) installed. To help your TA with this process, your program files must include:

1. A `requirements.txt` file that lists every external library your program relies on, such as `python_ta`, `pygame`, `numpy`, etc. You can adapt the following `requirements.txt` file we provided for CSC111:

```
# CSC111 Winter 2026: Python libraries we'll be using this
semester.

# Testing and code checking
hypothesis
pytest
python-ta

# Graphics and data visualization
plotly
pygame
pillow
```

> **Note**: do not list built-in libraries that come with Python, like `doctest`, `math`, or `random`. Only include ones that need to be installed separately. If you can't remember whether a module comes included with Python, look in the PyCharm settings window where you install new libraries (Python -> Interpreter). If you see the library listed in the table, it's external to Python and should be included in your `requirements.txt` file.

2. A Python module called `main.py` whose main block contains the code necessary for running your entire program. When run, this module should:

   1. Load the necessary files from the datasets (if applicable).
   2. Perform your computations on the data.
   3. Produce an output (which may or may not be interactive).

   Your `main.py` file can be quite short, and just import the necessary functions and data types from your other Python modules. To keep things simple, all of your Python modules should be kept in the same folder as `main.py`, and submitted together on MarkUs at the "top level" (no subfolders). We encourage you to define sample "runner" functions, similar to what we've done on course assignments.

Running `main.py` may produce an interactive way of interacting with your computational results. In this case, in your written report you should include instructions on how to use the interactive components.

You may, if you wish, provide a *few* different ways of calling your program's primary functions that are commented out in your `main.py` and that your TA should try individually. In this case, include clear comments inside your `main.py` file *and/or* in your written report to describe the different ways your TA can uncomment various function calls to run your program, and the differences they should look for in your program's output/results.

Important note about generating data

Your project might involve a long-running "data generation" or "training" phase. For example, in Assignment 2, we generated some of the provided game data with a separate Python program. In order for your program to get interesting results, you might find that you need to run this data generation for a long period of time, which isn't feasible for your TA to do.

If you are in this situation, make sure your submitted program allows your TA to do two things:

1. Run a short (under 1 minute) example of this data generation/training, so that your TA can verify that it works properly.
2. Save and load a "saved state" of the generated data or some other resulting data structure. Include this saved state in your submitted dataset zip file, and provide instructions for your TA to run your program to load this saved state, so that the rest of the program works well without needing to regenerate data.

Doing this should save you time as you are working on the project, too!

Evaluation

Below is an outline of the key criteria which will be considered during evaluation of your work:

Introduction & Project Goal Report Section

- This section should provide a clear **motivation** and all necessary background information for the project.
- The project question/goal should be **well-defined and specific** rather than broad or vague.
- A high degree of creativity and/or depth, and some external research, should be present.
- Inline citations should be used appropriately.

Dataset Description & Computational Overview Report Section

- Some projects may not require real-world datasets. But if used, all datasets should be **fully documented**.
- The **Computational Overview** should include:
  - How trees/graphs are used in the project (should be in a meaningful way).
  - A description of all key program components and the overall structure of the program.
- Writing should be clear and concise (mention all key details, but avoid sharing unnecessary/distracting details that would distract/confuse the reader from the main ideas).

Program Design & Code Quality

- We will look for **organization, readability, and style.**
- Code should follow a logical organization into files, classes and functions.
- Polish your code (remove “dead code”, eliminate code smells, etc.)
- Use clear, descriptive docstrings and names that adhere to standard style conventions.

Installing & Running the Program

- We should be able to install and run the program without any difficulty.
- Instructions should be straightforward and clear.
- Program output and interactions should be easy to understand and follow, based on provided instructions.

Description of Changes & Discussion Report Sections

- Both sections should clearly demonstrate evidence of reflection, analysis and critical thinking.
- Use specific examples and and details to illustrate points.
- Writing should be clear and concise.

Overall Computational Complexity

There are three key ways in which computational complexity can be demonstrated in your project:

1. Non-trivial dataset processing/model training. E.g., filtering/cleaning data, merging multiple datasets together programmatically, handling different file formats, generating trees/graphs to represent data.

2. Implementing non-trivial algorithms yourself. This goes beyond using built-in modules (e.g., statistics), or only calling functions from third-party libraries (e.g., sklearn), or going beyond what we did for exercises or on extra practice handouts. You may use things from other parts of the course as starting points, but must extend them in non-trivial ways to receive credit for this complexity in area.

3. Non-trivial reporting of and/or user interactivity with computational results. This may include: displaying multiple graphs (e.g., using plotly in a sophisticated way); AI-generated recommendations; displaying geographic data on a map; an interactive window or text-based shell (that involves **Python programming**) which allows a user to display different computational results.

A strong project (an A grade) will have at least two of the three elements listed above present in the project, or at least one of the elements present to an exceptional degree (creative, interesting, shows a lot of effort). An exceptional project (an A+) will have multiple elements present to an exceptional degree.

Peer Evaluation and Weighted Individual Grade

In this project, each group member will receive an individual grade that is based on both shared and individual contributions.

We are using a weighted marking system, which means your project will receive an overall grade, and your individual grade will be the project grade multiplied by your contribution score (out of 1).

Your contribution score will be determined based on your submission of a self- and peer-evaluation form (that is, each group member will submit a self-evaluation of their contribution as well as a peer-evaluation of the contribution for all other group members).

If everyone in your group contributed equally based on these evaluations, everyone's contribution score will be 1 and all individual marks will be the same as the overall project mark (individual mark = project mark * 1). If someone contributed significantly less than others based on multiple evaluations, they would have a contribution score below 1 leading to their individual mark being lower than the project mark.

Why Use Weighted Marks?

Many students find traditional group grading unfair because some members contribute more than others. The weighted system helps by:

- Rewarding the team as a whole.
- Recognizing individual efforts within the team.
- Preventing unfair distribution of work.

Contribution Score Criteria

Each team member will be evaluated based on the following four categories:

Communication

How often does this team member:

- Listen attentively to others without interrupting?
- Clarify ideas to ensure understanding?
- Explain their ideas clearly and concisely?
- Give good reasoning for their suggestions?
- Encourage team members and support group decisions?

Decision Making

How often does this team member:

- Analyze problems from multiple perspectives?
- Use logic in problem-solving?
- Offer well-reasoned solutions (rather than just guessing)?

- Solicit new ideas from the group?
- Adapt to changes and challenges?

Collaboration

How often does this team member:

- Encourage participation from all team members?
- Work towards solutions that benefit the whole group?
- Give credit to others for contributions?
- Accept constructive feedback without defensiveness?
- Cooperate effectively with the team?

Productivity

How often does this team member:

- Stay focused on tasks during meetings?
- Use meeting time efficiently?
- Set priorities for tasks and meet deadlines?
- Suggest ways to move the project forward?
- Monitor progress to ensure goals are met?

A form that you can use to submit these evaluations will be released after the project deadline passes.

Submission instructions

Warning: grace credits may not be used to extend the deadline of the course project proposal or final submission.

Please **proofread** and **test** your work carefully before your final submission. This final project is the culmination of a lot of work and learning you've done over the entire year, so take pride in it and check everything over before you submit!

Submitting files

1. Login to MarkUs (https://markus.teach.cs.toronto.edu/markus).

2. Go to *project2*, and the "Submissions" tab.

3. Submit the following files: `project_report.tex`, `project_report.pdf`, `requirements.txt`, `main.py`, plus *all* your other project files.

   Please note that MarkUs is picky with filenames, and so you must submit filenames that exactly match the four listed, including using lowercase letters. You can choose the names of the other Python files you submit, but make sure to follow standard Python naming conventions (lowercase letters, with words separated by underscores).

4. Test your program out on a *fresh computer* to check that everything will work properly for your TA.

   Here are the steps to follow:

   a. If you don't have a fresh computer, first *uninstall Python 3.13.5* and then re-install it. If you have a fresh computer, install Python 3.13.5 and PyCharm on it (using the instructions on Quercus (https://q.utoronto.ca/courses/418277/pages/software-installation-for-csc111)).
   b. Download all of the files you submitted to MarkUs onto the computer, in a new folder.
   c. Install all of the Python libraries listed under the `requirements.txt` file you submitted.
   d. Follow any additional installation instructions you wrote in your project report.
   e. Download all required datasets by following the instructions you wrote in your project report.
   f. Run your `main.py` file (e.g., in PyCharm). **It should "just work"**—if you run into any errors, fix them and (if necessary) update your report's instructions so that your TA doesn't run into problems!!
   g. Review the output/product of your program, and the descriptions/instructions you wrote in your report for interpreting the results or interacting with your program. Your TA should be able to make sense of them!

You can also test out your instructions with a family member or friend (not in CSC111)—they should be able to follow all of these required steps even if they don't have any technical knowledge, except possibly needing help with "running" `main.py`. Plus it's a good chance to show off your work! 😎

Remember, you can submit your files multiple times before the due date. So you can aim to submit your work early, and if you find an error or a place to improve before the due date, you can still make your changes and resubmit your work.

After you've submitted your work, please give yourself a well-deserved pat on the back and go take a rest or do something fun or eat something tasty! (with your project team!) **You've completed your last course project for CSC111!**

Strawberry showing that good projects deserve both quiet contemplation and loud celebration